

Lakehouse cho LLM observability ở 1 tỷ request/ngày

Đỗ Quốc An | MSSV 2A202602892 | K4-Track02-Day18 | Topic A

1. Bài toán và ràng buộc

Thiết kế giả định nhận 1 tỷ request/ngày, mỗi request 5 KB theo đơn vị thập phân: 5 TB raw/ngày, trung bình 11.574 request/giây; sizing peak bằng 3 lần trung bình. Dashboard theo tenant cần dữ liệu mới trong 5 phút. Prompt/response đã redact được giữ 7 ngày; sau đó chỉ giữ aggregate 1 năm. Không cho người dùng đọc PII trước redact. Storage cap là 5.000 USD/tháng; compute được tính riêng, không giả vờ nằm trong cap này. Mục tiêu khó nhất là vừa xóa hết bản sao nội dung hết hạn vừa giữ đủ lịch sử để reader và recovery an toàn. Đây là đề xuất, chưa phải benchmark production hay chứng nhận tuân thủ pháp luật.

2. Kiến trúc và luồng dữ liệu

SDK/gateway: redact + tokenize trước khi ghi, schema contract
| request_id, tenant token, ingest/event timestamp
v

Durable queue (6h buffer, service identity; không giữ raw PII)
| 60s micro-batch, checkpoint offsets
v

BRONZE Delta: redacted envelopes, partition ingest_day
| validate + dedup request_id; ACID commits
v

SILVER Delta: typed events, partition ingest_day
| hourly compact 128-256 MB; Z-order tenant token
| incremental aggregates; idempotent window overwrite
v

GOLD Delta: tenant/model/day/5min metrics, retention 365 ngày
| approved views + tenant filter
v

Dashboard API/cache: freshness watermark <= 5 phút

Registry/catalog + ACL: owner, locations, schema, version/run manifest

Maintenance controller: reader leases + delete partitions + VACUUM

Audit/metrics: input/output counts, quality, queue lag, reclaimed bytes

Medallion được dùng để tách raw envelope đã redact, sự kiện chuẩn hóa và số liệu phục vụ dashboard. ACID bảo vệ từng commit của mỗi bảng; không giả định transaction xuyên Bronze/Silver/Gold. Một run manifest ghi offset range và version từng bảng để xác định frontier nhất quán. Z-order dùng tenant token cho query incident có predicate tenant; checkpoint giảm log replay. Time travel phục vụ rollback trong cửa sổ được giữ, không hứa khôi phục dữ liệu đã VACUUM. Catalog là điểm tra cứu bảng và policy; object-store IAM vẫn là security boundary riêng.

3. Quyết định và phương án bị loại

D1 - Table format: chọn Delta. CDF và MERGE phục vụ dedup, sửa late event, cập nhật aggregate; stack Spark trong thiết kế phải được kiểm thử riêng với lab delta-rs. Loại plain Parquet vì thiếu atomic log, version và audit cho rewrite. Loại Iceberg cho MVP này vì team đã có pipeline Delta/CDF; đổi format cần kiểm lại semantics delete và consumer checkpoints. Iceberg vẫn phù hợp khi ưu tiên nhiều engine và REST catalog; lựa chọn này là theo năng lực team, không khẳng định Delta luôn tốt hơn. [Nguồn: Delta CDF.](#)

D2 - Catalog: chọn registry/catalog tương thích Delta với service identity và location allowlist. Một job chỉ resolve logical table name sang location đã cấp quyền; run manifest pin version. Loại hardcoded đường dẫn S3 ở từng dashboard vì đổi location/owner làm nhiều consumer drift. Loại tự động chuyển Delta sang Iceberg REST catalog vì REST catalog của Iceberg không tự cung cấp transaction semantics Delta; migration không phải một đổi config. MVP dùng registry nhỏ và IAM role; production phải thử ACL, audit và recovery. [Nguồn: Iceberg REST catalog.](#)

D3 - Layout: chọn partition ingest_day + cluster tenant trong ngày. Ingest time làm retention dễ kiểm; event time vẫn là cột để tính Gold. File target 128-256 MB, không mở partition riêng cho từng tenant. Loại partition tenant_id vì 10.000 tenant sinh nhiều partition nhỏ; tenant ít traffic gây metadata/file overhead. Loại chỉ partition theo event_day vì late event có thể mở lại ngày đã hết hạn và phá cutoff; sẽ áp dụng admission rule trước khi ghi. Query event time phải kèm ingest bounds hoặc manifest cho incident. Trước khi bật Z-order ở scale, đo rewrite bytes và skipping trên workload tenant.

D4 - Compression: chọn Parquet ZSTD với budget 3:1 cho redacted JSON và 4:1 cho typed Silver. Đó là giả định sizing cần benchmark, không phải kết quả lab. Loại uncompressed vì 35 TB payload cho mỗi lớp làm tăng storage/scan. Loại gzip JSON làm format analytical vì khó projection pushdown và schema enforcement. ZSTD có thể tốn CPU hơn codec nhẹ; acceptance yêu cầu throughput peak vẫn đạt và query không vượt budget. Nếu ratio chỉ 1:1, storage vẫn cần được tính lại cùng queue và file rewrite headroom.

D5 - Lifecycle: chọn logical retention 6 ngày 23 giờ + tối đa 1 giờ cleanup. Cutoff xét ingest timestamp; queue loại bỏ mọi event quá TTL trước retry. Payload quá 7 ngày không được giữ trong log, backup, dead-letter queue hoặc file tombstoned. Reader lease tối đa 15 phút; job cleaner chờ lease hết, rồi xóa partition và VACUUM theo retention được kiểm chứng. Loại mặc định giữ mọi tombstone 7 ngày sau logical delete vì physical payload có thể tồn tại gần 14 ngày. Loại VACUUM 0 tùy tiện khi writer/reader đang chạy vì có thể làm mất dữ liệu đang dùng. Với yêu cầu incident đủ đúng 7 ngày từng giây, cần thương lượng cleanup SLA; thiết kế hiện cung cấp gần 7 ngày và deadline physical 7 ngày, không hứa cả hai tuyệt đối.

D6 - Governance: chọn redact ở SDK/gateway trước durable queue và schema quarantine không chứa payload nhạy cảm. Tenant dùng token; mapping token ở service riêng, quyền hạn chế, audit mỗi lần incident lookup. Loại redact chỉ ở Gold vì Bronze vẫn có PII đọc được. Loại regex đơn lẻ rồi gắn nhãn 'đã an toàn' vì tên/ngữ cảnh và scan có thể lọt; cần allowlist fields, detector test và manual review mẫu đã hạn chế quyền. Đây là cơ chế kỹ thuật đề xuất, không kết luận đạt một luật cụ thể.

D7 - Dashboard: chọn Gold cập nhật theo cửa sổ và cache có watermark. Loại scan Bronze 5 TB/ngày mỗi lần refresh vì bytes và parse cost tăng cùng traffic. Loại aggregate bằng append không idempotent vì retry làm double-count. Ghi đè window bị ảnh hưởng bằng job single-writer, dedup trước aggregate, rồi publish frontier; late events trong 24 giờ tính lại window. Event quá retention chỉ vào audit counter, không tái tạo payload cũ. p95 toàn ngày tính từ histogram/sketch hoặc Silver còn giữ; không lấy trung bình p95 của các window.

4. Sizing và phép tính chi phí

Đơn vị: TB = 10^{12} bytes, GB = 10^9 bytes; tháng 30 ngày. Mọi đơn giá dưới đây là **budget input giả định**, không phải báo giá AWS hay nhà cung cấp ngày hiện tại. Cần thay bằng báo giá region và benchmark trước khi duyệt production.

Bronze: 5 TB/ngày / 3 x 7 = **11,67 TB**.

- Silver gồm typed metrics và bản redacted payload phục vụ incident: giả định 4 TB raw/ngày / 4 x 7 = **7 TB**; không gọi đây là tổng storage chỉ vì bỏ Bronze.
- Gold: 10.000 tenant x 3 model x 288 window/ngày x 365 x 100 bytes/row = **315,36 GB** đã budget nén; histogram/keys phải nằm trong 100 bytes hoặc tăng sizing.
- Tổng current payload = 11,6667 + 7 + 0,31536 = **18,982 TB**.
- Dự phòng tombstone, rewrite và metadata 50% = 9,491 TB; tổng billed lakehouse **28,473 TB**.
- Queue 6 giờ chứa redacted input: 5 TB/ngày / 4 = **1,25 TB**, không tính lại vào Bronze headroom. Budget queue storage 100 USD/TB-tháng = **125 USD/tháng**.
- Lakehouse storage: 28,473 x 25 USD/TB-tháng = **711,83 USD/tháng**.
- Audit/control budget 0,1 TB x 25 = **2,50 USD/tháng**.
- Tổng storage giả định **839,33 USD/tháng**, thấp hơn cap 5.000; phần còn lại dành sensitivity và sai số, không phải bảo đảm tài chính.

Sensitivity: nếu Bronze và Silver đều chỉ nén 1:1, current = 35 + 28 + 0,31536 = 63,315 TB; x1,5 x25 +125+2,5 = **2.501,83 USD/tháng**. Với mức giá lakehouse giả định gấp đôi (50/TB-tháng), worst case này thành **4.876,15 USD/tháng**. Nếu queue retention tăng hoặc payload >5 KB thì có thể vượt cap; budget controller phải phát hiện, không tự xóa dữ liệu sớm để làm đẹp số.

Compute riêng: ingress 16 vCPU x 0,05 USD/vCPU-giờ x720 = **576**; transform 64 x0,05 x720 = **2.304**; compaction 8 x0,05 x4 giờ/ngày x30 = **48**; query/API 8 x0,05 x720 = **288**; catalog/audit allowance **100**. Compute tổng **3.316 USD/tháng**; nếu request operations 20 triệu PUT x0,005/1000 +100 triệu GET x0,0004/1000 = **140**, tổng scenario storage+compute+requests **4.295,33 USD/tháng**. Network, managed queue compute, KMS, support, tax và multi-region replica chưa được quote; phải cộng riêng để ra TCO đầy đủ.

Thông lượng Silver: 4 TB raw/ngày /86.400 = 46,3 MB/giây; peak x3 =139 MB/giây. 64 vCPU phải đạt ít nhất 2,2 MB/giây/vCPU trước overhead; đây là điều kiện benchmark. Với 1 TB/ngày compressed và target256 MB, khoảng 3.906 file/ngày Silver; micro-batch nhỏ vẫn cần compaction. Freshness budget 60s queue +120s transform +60s aggregate +60s API =300s. Không suy ra p95 serving từ notebook toy hoặc phép tính storage.

5. Failure modes, phát hiện và phục hồi

F1 - Duplicate/replay sau crash giữa Silver và Gold. Detect bằng offset-range manifest, request_id unique count, so Gold totals với Silver ở cùng frontier. Rollback Gold về version tốt hoặc tính lại window idempotent từ Silver pin version; không rewind source offsets mù quáng. Delta ACID không làm ba bằng commit cùng lúc.
Nguồn: [concurrency](#).

F2 - Schema drift đổi latency thành string. Detect schema contract tại ingress và enforcement Silver; alert quarantine rate >0, không bỏ assertion. Freeze publish watermark, sửa adapter và replay redacted queue; schema_mode merge chỉ dùng khi approved additive field. Không cho auto-evolution thay dữ liệu sai kiểu thành NULL im lặng.

F3 - VACUUM xóa file reader/CDF consumer còn cần. Detect consumer lag, active reader lease và preflight version availability. Hoãn cleanup khi lease hợp lệ; nếu cần tuân TTL thì stop reader, deny old query rồi cleanup theo policy. Sau file đã bị xóa không hứa RESTORE cứu được; rebuild dữ liệu trong TTL từ queue/current parent, sự kiện hết TTL phải được báo mất recovery. CDF có retention, không phải audit vĩnh viễn. Nguồn: [CDF retention](#).

F4 - Crashed writer tạo orphan và bill tăng. Detect disk/object inventory minus file references của **mọi version còn giữ**, không chỉ current, với age guard >max writer duration. Scratch lab cho thấy delta-rs vacuum không bắt orphan chưa commit. Cleaner production chỉ sweep prefix job-owned sau lease/age check; thử dry-run, audit danh sách và canary table trước rollout. Rollback là dừng sweep khi reference check không đủ; không có phép undo magic sau xóa vật lý. Nguồn: [utility/retention](#).

F5 - Detector bỏ lọt PII. Canary synthetic PII và sampling gate phát hiện trước publish; incident revoke access tới partition liên quan, dừng ingress adapter và regenerate redacted payload trong TTL. Không RESTORE version đã rò rỉ ra dashboard. Audit xác định consumer chịu ảnh hưởng, invalidation cache theo manifest. Cơ chế này cần review security riêng.

F6 - Source lag hoặc aggregate sai p95. Detect watermark >300s, input/output counters và so sketch với exact quantile trên sample. Rollback dashboard tới frontier trước kèm stale indicator; sửa aggregate rồi backfill 24h có version pin. Không che freshness bằng cách cập nhật timestamp của cache mà dữ liệu vẫn cũ.

6. MVP một tuần và kiểm tra tính khả thi

Ngày 1: schema, 1 triệu synthetic requests/100 tenant, redaction canary và input contracts. Ngày 2: queue replay + Bronze Delta, offset manifest, fault injection trước/sau commit. Ngày 3: Silver MERGE/dedup, late event 24h, schema bad-write tests. Ngày 4: Gold window recompute, histogram p95, API watermark. Ngày 5: compaction/Z-order, stats skipping, catalogue registry/IAM test. Ngày 6: accelerated TTL lab với lease reader, crash orphan và CDF consumer lag. Ngày 7: load test, cost report, design review và production go/no-go.

Acceptance: 0 duplicate request_id Silver sau replay; Gold counts/cost khớp recompute; $p50 \leq p95$; cache watermark $\leq 300s$ khi peak; canary PII không xuất hiện ở queue/bảng/API; target compaction $\geq 10x$ và skip $\geq 50\%$ trên query tenant; tất cả version đang được pin đọc được trước lease expiry; sau deadline cleanup không tìm thấy payload marker ở active data, tombstones, orphan, queue và test backup. Benchmark peak 34.722 request/giây và burst payload qua generator/load tool riêng; chưa có bằng chứng thì chưa duyệt production.

Mechanism khó nhất là **retention vật lý trong khi reader/CDF còn hoạt động**. PoC thu nhỏ TTL và dùng marker duy nhất; giữ reader lease, crash writer tạo orphan, chạy preflight và xác nhận cleaner từ chối. Cho lease hết, consumer checkpoint tiến, logical delete rồi sweep/vacuum, kiểm cả metadata reference và bytes trên disk. Không dùng lab NB6 retention 0 làm cấu hình production. Một PoC riêng chưa được thực thi trong bản brief này; các notebook NB1-NB8 đi kèm chỉ xác minh cơ chế lab ở scale nhỏ. Deployment, latency peak, compression và forecast cost còn là điều kiện nghiệm thu.

7. Nguồn và phạm vi AI

Đề bài: docs/bonus/BONUS-CHALLENGE.md; cơ chế minh họa: NB2, NB4, NB6, NB8 và output trong submission/notebooks. Tài liệu chính thức liên kết tại từng quyết định/failure mode, truy cập 04/10/2026. Đơn giá đều giả định để kiểm phép tính; không lấy bằng giá lab làm quote dịch vụ.

Codex hỗ trợ soạn cấu trúc, phân tích alternatives, phép tính và đối chiếu nguồn. Người nộp cần tự đọc, điều chỉnh theo hiểu biết của mình và bảo vệ thiết kế khi coach hỏi; khai chi tiết ở ../AI_USAGE.md.